Android 多媒体
开发指南

版本号: 1.1
发布日期: 2023.11.16

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.03.15	AWA1438	初始版本文档。
1.1	2023.11.16	AWA1782	1. 增加 Codec2.0 相关描述和框图。 2. 添加全志 Codec2.0 对接层代码目录结构。 3. 添加 MediaCodec 通路播放问题调试方式。 4. 修改为使用配置文件来动态保存码流和解码数据。

广州星翼电子科技有限公司zdyzlinux

目　录

1 前言 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1
1.4 相关术语 1

2 多媒体模块框架和支持列表 **2**
2.1 Android 多媒体框架 2
2.2 代码结构 3

3 多媒体支持说明 **7**
3.1 视频容器格式支持说明 7
3.2 视频解码格式支持说明 7
3.3 音频容器格式支持说明 7
3.4 音频解码格式支持说明 7
3.5 流媒体协议支持说明 7
3.6 多屏互动支持说明 7
3.7 支持第三方使用 MediaCodec 的播放器 8
3.8 支持多音轨切换 8
3.9 支持的其他播放特性 8

4 模块配置使用说明 **9**
4.1 最后一帧显示黑屏配置说明 9
4.2 开机视频、动画使用说明 9
4.2.1 开机视频使用说明 9
4.2.2 开机动画使用说明 9
4.2.3 客户开机动画视频定制使用说明 10
4.3 开机音乐使用说明 11
4.3.1 开机音乐使用说明 11
4.3.2 配置 audio_conf 11

5 调试方法 **12**
5.1 如何修改多媒体中间件打印等级 12
5.2 如何简单定位 MediaPlayer 播放问题 12
5.2.1 不能播放问题的定位 12
5.2.2 播放异常定位 13
5.3 如何简单定位 MediaCodec 通路播放问题 15
5.4 如何保存解码前的码流和解码后的图片 16

6 FAQ **17**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.1 为什么有些音视频不能播放 17

广州星翼电子科技有限公司zdyzlinux

1 前言

1.1 编写目的

为了让多媒体开发人员熟悉 Android 13 产品的多媒体框架，实现多媒体功能定制和简单调试。

1.2 适用范围

本模块说明适用于全志科技 Android 13 系统产品, 其他 Android 版本系统也可参考。

1.3 相关人员

多媒体开发人员。

1.4 相关术语

- CedarX：全志多媒体中间件，通过 AwPlayer 对接到 Android 系统中，架构详见 “2.1 Android 多媒体框架” 章。
- CedarC：全志多媒体视频编解码驱动以及 openMAX IL 层实现。
- Stream：CedarX 对多媒体协议类型访问的统一接口，支持的媒体协议类型包括：本地文件、文件描述符、RTSP、HTTP、SSL、TCP、RTMP、MMS、MMSH、MMST、MMSHTTP 等。
- Parser：CedarX 对封装格式的解析的统一接口，支持的媒体封装类型包括：ASF、TS、AVI、FLV、MKV、MOV、DASH、RTSP、HLS、PMP、OGG、MPG、MMS、MMSHTTP、M3U9、PLAYLIST、MP3、APE、FLAC、AMR、ATRAC、REMUX 等。
- Demuxer：CedarX 对媒体的 Stream 和 Parser 解析的统一接口。
- Decoder：音频，视频，字幕的解码器。
- Render：音频，视频，字幕渲染。
- Codec2.0: 谷歌在 Android Q 上推出了 Codec2.0，指在于取代 ACodec 与 OpenMAX，它可以看作是一套新的对接 MediaCodec 的中间件，往上对接 MediaCodec Native 层，往下提供新的 API 标准供编解码使用，相当于 ACodec 2.0。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 多媒体模块框架和支持列表

2.1 Android 多媒体框架

Android 原生多媒体框架基于 MediaPlayer 播放器通路，以及全志科技的移植框架图，下图中 CedarX 是全志多媒体中间件。

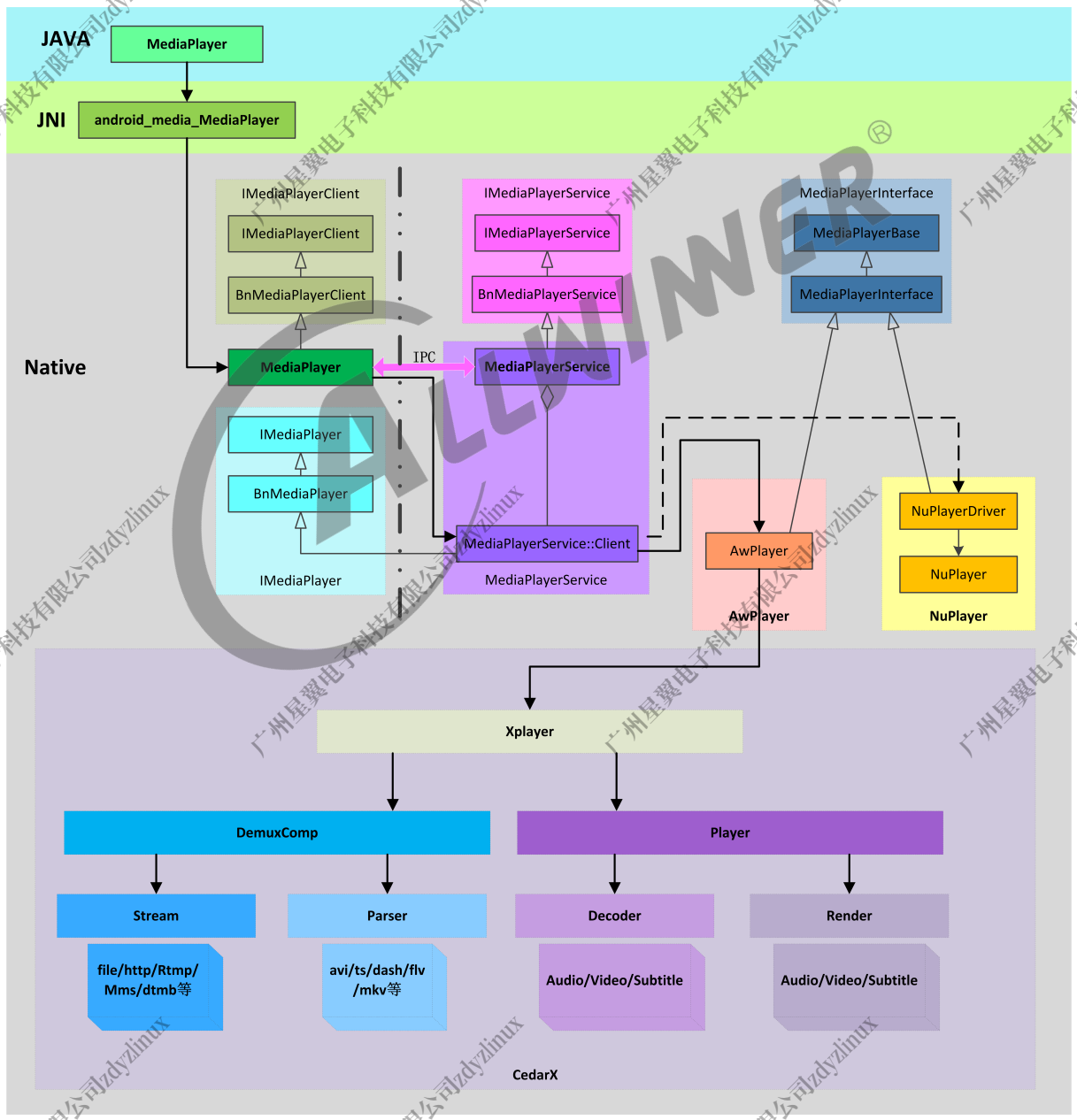

图 2-1: Android CedarX flows

Android 原生多媒体框架基于 MediaCodec 通路，从 android12 起，虚线部分的 ACodec 部分和 OpenMax 开始被废弃，取而代之的是 CCodec 和 Codec2.0 框架。

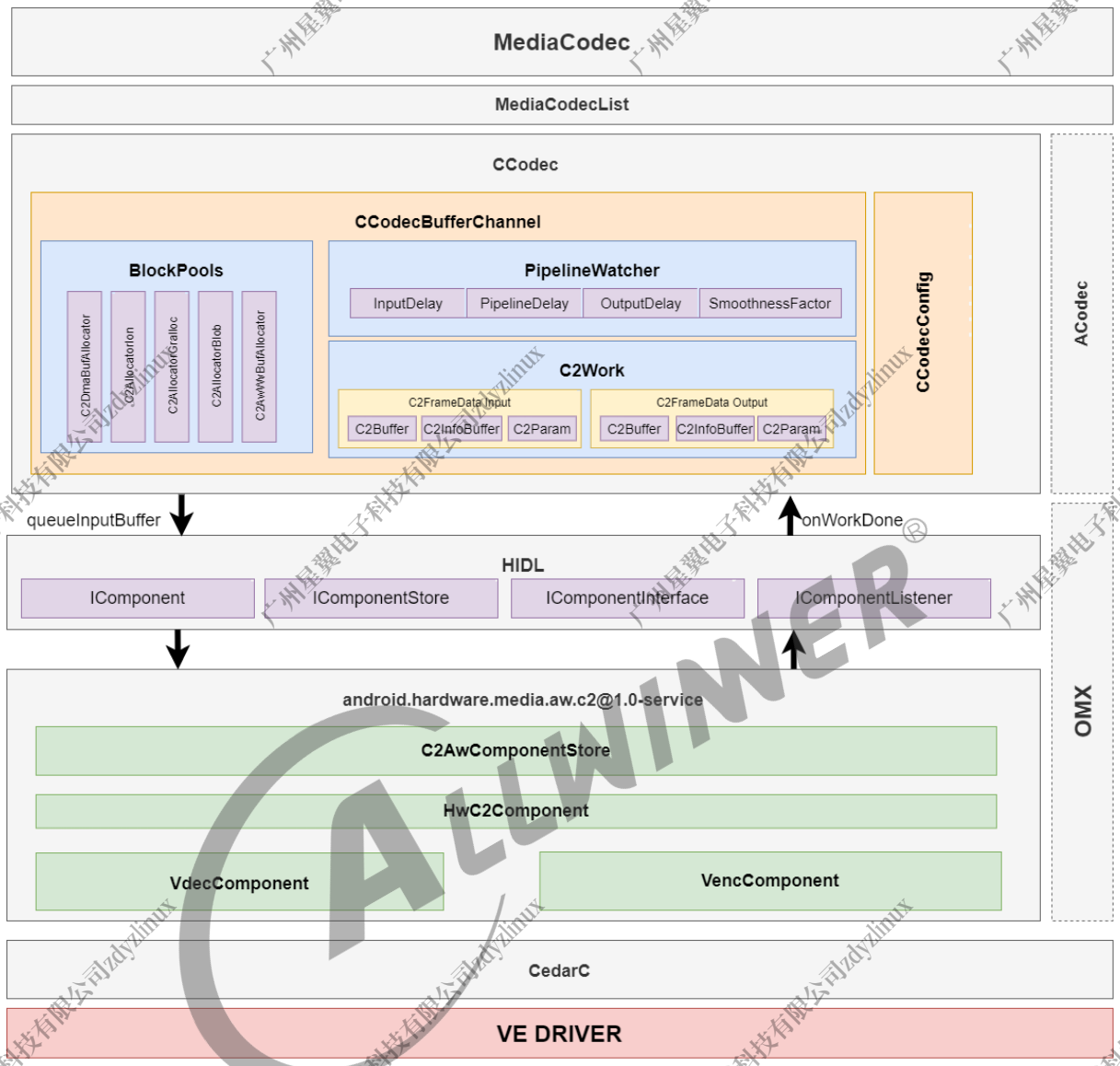

图 2-2: Android MediaCodec flows

2.2 代码结构

1、Android 多媒体模块，java 层和 jni 层代码目录。

```
android/frameworks/base/media
 ├── aidl
 ├── aidl_api
 ├── Android.bp
 ├── java
 ├── jni
 ├── lib
 ├── mca
```

```
├── native
├── OWNERS
├── packages
├── TEST_MAPPING
└── tests
```

2、Android 多媒体模块，Native 层代码目录。

```
android/frameworks/av/media
├── Android.mk
├── audioserver
├── bufferpool
├── codec2
├── codecs
├── common_time
├── extractors
├── img_utils
├── janitors
├── libaaudio
├── libaudioclient
├── libaudiofoundation
├── libaudiohal
├── libaudioprocessing
├── libcedarc
├── libcedare
├── libcedarx
├── libcpustats
├── libdatasource
├── libeffects
├── liberror
├── libheadtracking
├── libheif
├── libmedia
├── libmediaformatshaper
├── libmediahelper
├── libmediametrics
├── libmediaplayerservice
├── libmediatranscoding
├── libnbaio
├── libnblog
├── libshmem
├── libstagefright
├── libwatchdog
├── mediaserver
├── mtp
├── ndk
├── OWNERS
├── TEST_MAPPING
├── tests
└── utils
```

3、CedarX 多媒体中间件目录。

```
android/frameworks/av/media/libcedarx
├── android_adapter
│   ├── awplayer
│   ├── base
│   ├── iptv
│   ├── metadataretriever
│   └── output
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
├──── awrecorder
├──── config
├──── demo
├──── external
├──── libcore
│   ├──── base
│   ├──── common
│   ├──── muxer
│   ├──── parser
│   ├──── playback
│   └──── stream
├──── xmetadataretriever
└──── xplayer
```

4、CedarC 多媒体编解码库目录。

```
android/frameworks/av/media/libcedarc
├──── base
├──── config
├──── include
├──── library
├──── memory
├──── openmax //在android13开始，openmax已经被codec2取代
│   ├──── include
│   ├──── libstagefrighthw
│   ├──── omxcore
│   ├──── vdec
│   └──── venc
├──── vdecoder
└──── vencoder
```

5、Codec2.0 对接层代码目录。

```
├──── aidl
│   ├──── aidl_api
│   ├──── Android.bp
│   ├──── default
│   └──── vendor
├──── Android.bp
├──── components
│   ├──── Android.bp
│   ├──── C2DebugHelper.cpp
│   ├──── decoders
│   ├──── encoders
│   ├──── exports.lds
│   ├──── HwC2Component.cpp
│   ├──── HwC2Interface.cpp
│   ├──── include
│   ├──── neon_rgb2yuv.s
│   ├──── VdecComponent.cpp
│   └──── VencComponent.cpp
├──── config
│   ├──── Android.bp
│   └──── c2codec_config.go
├──── plugins
│   ├──── Android.bp
│   └──── widevine_plugins
├──── services
│   ├──── Android.bp
│   ├──── android.hardware.media.aw.c2@1.0-service.rc
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    ├──── Android.mk
    │   ├──── manifest_media_c2_V1_0_default.xml
    │   ├──── seccomp_policy
    │   └──── vendor.cpp
    └──── vndk
        ├──── Android.bp
        ├──── C2Store.cpp
        └──── include
```

广州星翼电子科技有限公司zdyzlinux

3 多媒体支持说明

3.1 视频容器格式支持说明

A 系列和 T 系列默认支持如下封装格式：asf, avi, flv, f4v, mkv, mov, mp4/m4v, vob, mpg, pmp, ts/tp, m2ts, mts, wmv, webm, 3GP。

3.2 视频解码格式支持说明

A 系列和 T 系列默认支持如下视频格式：H.265 MP/L5.2、H.264 Baseline/HP/MP Level5.1、MPEG1/MPEG2/MPEG4、H.263 Baseline 等。

3.3 音频容器格式支持说明

A 系列和 T 系列默认支持如下音频封装格式: aac, aiff, amr, ape, atrac, caf, dsd, flac, g729, mp3, ogg, wav 等。

3.4 音频解码格式支持说明

A 系列和 T 系列默认支持如下音频解码格式: AMR, MP1/MP2/MP3, OGG, WAV, AAC, APE, FLAC, DSD, G729, ALAC 等。

3.5 流媒体协议支持说明

A 系列和 T 系列默认支持如下流媒体协议：http、https、rtmp、hls。

3.6 多屏互动支持说明

多屏互动即 Miracast 功能，是由 Wi-Fi 联盟于 2012 年所制定，以 Wi-Fi 直连为基础的无线显示标准。支持此标准的设备可通过无线方式分享视频画面，例如手机可通过 Miracast 将影片或照片直

版权所有 © 珠海全志科技股份有限公司。保留一切权利

接在电视或其他装置播放而无需受到连接线缆长度的影响。

A 系列支持 Miracast 多屏互动协议，包括发送端和接收端。

T 系列仅支持接收端。

3.7 支持第三方使用 MediaCodec 的播放器

A 系列和 T 系列默认支持使用 Android 原生标准接口 MediaCodec 的视频播放 apk，比如 Kodi 等。

3.8 支持多音轨切换

A 系列和 T 系列默认支持多音轨切换。

3.9 支持的其他播放特性

A 系列和 T 系列的 H264 和 H265 最大播放规格支持如下：

- H.265 4K@60fps@10bit
- H.264 4K@30fps@8bit

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块配置使用说明

4.1 最后一帧显示黑屏配置说明

最后一帧显示黑屏是指在切台场景，切换中屏幕保持黑屏。修改对应方案的 CedarX 配置文件：android/frameworks/av/media/libcedarx/config/cedarx_config.go。

```
var xxx_cflags = []string {
  "-DCONF_SEND_BLACK_FRAME_TO_GPU",
  ...
```

此功能配置默认关闭。

4.2 开机视频、动画使用说明

4.2.1 开机视频使用说明

将视频命名为 boot.mp4，放到/system/media/下或者/data/local/下。系统启动优先从/data/local/检测视频文件，如果没有则从/system/media/下获取，如果两个路径件都没有，则使用默认启动动画。

4.2.2 开机动画使用说明

1. 文件存放位置

开机画面都是以附件 bootanimation.zip 的形式存放在机子中。存放位置有两个：

- /system/media/bootanimation.zip
- /data/local/bootanimation.zip

读取顺序：机子开机画面或视频，优先读取/data/local/bootanimation.zip，如果发现这个 zip 包没有或者解压有问题或者包中的文件有异常或损坏，则使用/system/media/bootanimation.zip 作为开机画面或开机视频展现。

2. 开机画面的内部结构

版权所有 © 珠海全志科技股份有限公司。保留一切权利

名称	大小	压缩后大小
part1	784 755	784 755
part0	2 946 219	2 946 219
desc.txt	38	38
audio_conf.txt	565	565

图 4-1: bootanim01

文件夹中存放图片，part0 内容。

1_00000.png	2017/2/19 14:42	PNG 图片文件	7 KB
1_00006.png	2017/2/19 14:42	PNG 图片文件	9 KB
1_00015.png	2017/2/19 14:42	PNG 图片文件	14 KB
1_00024.png	2017/2/19 14:42	PNG 图片文件	24 KB
1_00030.png	2017/2/19 14:42	PNG 图片文件	32 KB
1_00036.png	2017/2/19 14:42	PNG 图片文件	36 KB
1_00042.png	2017/2/19 14:42	PNG 图片文件	33 KB
1_00048.png	2017/2/19 14:42	PNG 图片文件	32 KB
1_00054.png	2017/2/19 14:42	PNG 图片文件	31 KB
1_00060.png	2017/2/19 14:42	PNG 图片文件	30 KB
1_00072.png	2017/2/19 14:42	PNG 图片文件	28 KB
1_00084.png	2017/2/19 14:42	PNG 图片文件	33 KB
1_00096.png	2017/2/20 19:24	PNG 图片文件	47 KB
audio.wav	2017/2/24 11:20	波形声音	1,936 KB

图 4-2: bootanim02

/desc.txt 文件及内容分析。

```
840 840 12
p 1 0 part0
p 0 0 part1
```

图 4-3: bootanim03

840 840 是指前面文件夹里 png 的分辨率，12 是指每秒播放帧数；p 是标识符，1 0 两个数字分别指循环次数和阶段间隔时间，0 0 就代表循环播放；part0 就是文件夹的名字，文件夹的名称和存放图片的目录名一致，设计结构：上图/desc.txt 文件设置为：part0 播放一次，间隔 5 秒，part1 循环播放。

4.2.3 客户开机动画视频定制使用说明

设置或者更新开机动画视频时，客户需要自行实现开机动画视频文件下载逻辑（开机动画 bootanimation.zip 或开机视频 boot.mp4），把文件存放到制定目录即可。如果已经有开机动画视频文

件，覆盖即可。

4.3 开机音乐使用说明

4.3.1 开机音乐使用说明

开机音乐代码位于 frameworks/base/cmds/bootanimation 下，会将 bootanimation.zip 中打包的 wav 音频直接播放出来。bootanimation.zip 在 sdk 中的存放目录是 device/softwinner/[platform_name]/，在小机的存放目录是/system/media/。

4.3.2 配置 audio_conf

在 frameworks/base/cmds/bootanimation 中的 BootAnimation.cpp 中，预留了 audioplay 的服务，可以直接播放 wav 格式的数据，前提是配置好 audio_conf.txt。配置文件 audio_conf.txt 同样打包在 bootanimation.zip 中，该文件会配置具体音频路由通路，包含喇叭和耳机通路。audio_conf.txt 如下所示，其中 mixer 表示喇叭配置，headset mixer 表示耳机的配置。

```
card=0
device=0
period_size=2048
period_count=4
#Input_Event or Switch_State
Headset detection=Switch_State
switch_path=/sys/module/snd_soc_sunxi_component_jack/parameters/jack_state
single_volume_control=1
speaker_headphones_out=0
#mixer "Headphone Volume"=0
#mixer "LINEOUT volume"=26
mixer "LINEOUT Gain"=15
mixer "LINEOUT Output Select"=single
mixer "LINEOUTL Switch"=1
mixer "LINEOUTR Switch"=1
mixer "SPK Switch"=1
#headset mixer "Headphone Volume"=0
#headset mixer "LINEOUT volume"=26
headset mixer "HPOUT Gain"=4
headset mixer "SPK Switch"=0
headset mixer "HPOUT Switch"=1
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 调试方法

5.1 如何修改多媒体中间件打印等级

针对多媒体音视频播放问题，客户可以修改 CedarX 配置文件打印等级，增加相关打印信息，默认为 DEBUG 级别：android/frameworks/av/media/libcedarx/conf/saturn_cedarx.conf。

```
# log will output if level >= log_level
#VERBOSE = 2,
#DEBUG = 3,
#INFO = 4,
#WARNING = 5,
#ERROR = 6,
log_level = 3
```

5.2 如何简单定位 MediaPlayer 播放问题

遇到播放问题，可以先将 log_level 改为 VERBOSE，反馈给 FAE。

5.2.1 不能播放问题的定位

第一步：播放器有没有创建成功。

查看 adb logcat，搜集 CedarX 相关打印，可以过滤关键字 awplayer，出现如下信息表示播放器正常创建。

```
D/awplayer: <LogVersionInfo:34>:
D/awplayer: >>>>>>>>>>>>>>>>>>>>>>>>>>> CedarX <<<<<<<<<<<<<<<<<<<<<<<<<<<
D awplayer: tag  : CedarX-2.8.0
D awplayer: branch: master
D awplayer: commit: 967535b8ff6a073cb4f38e85a4ae5fa6008014d8
D awplayer: date  : Mon, 15 May 2017 01:30:22 +0000 (09:30 +0800)
D awplayer: author:
D awplayer: ----------------------------------------------------------------
```

查看 adb logcat，搜集 CedarC 打印。

```
I cedarc  : >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>> Cedar Codec <<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
I cedarc  : tag  : CedarC-v1.3.0
I cedarc  : branch: stable_v1.3.0_common
I cedarc  : commit: dedc20c57e6626caf25e0fa3042be4c8faf13e0f
I cedarc  : date  : Fri Nov 13 10:19:03 2020 +0800
I cedarc  : author: jenkins8080
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
I cedarc : patch :
I cedarc : ----------------------------------------------------------------
```

第二步：parser 有没有解析成功，过滤关键字 demuxComponent。

```
D/demuxComponent( 1515): <PrintMediaInfo:426>: *****PrintMediaInfo begin*****
D/demuxComponent( 1515): <PrintMediaInfo:440>: fileSize = 47174592, bSeekable = 1,
duration = 202996, audioNum = 1, videoNum = 1, subtitleNum = 0
D/demuxComponent( 1515): <PrintMediaInfo:458>: **Video[0]** eCodecFormat = 0x115,
nWidth = 720, nHeight = 480, nFrameRate = 24000,
nFrameDuration = 0, bIs3DStream = 0
D/demuxComponent( 1515): <PrintMediaInfo:476>: ***Audio[0]*** eCodecFormat = 0x3,
eSubCodecFormat = 0x0, nChannelNum = 2,
nBitsPerSample = 0, nSampleRate = 44100
D/demuxComponent( 1515): <PrintMediaInfo:492>: ****PrintMediaInfo end*****
```

没有如上类似的打印，说明 parser 没有解析成功。如果需要的话，搜集 CedarC 和 CedarX 打印，反馈给 FAE 定位问题。

5.2.2 播放异常定位

播放的处理流程如下。

```
        ——> AudioDecoder ——> AudioRender
Demuxer ——|
        ——> VideoDecoder ——> VideoRender
```

播放异常的分析流程与之相反：Render —> Decoder —> Demuxer。

1、查看 Audio/Video Render 是否正常

android/frameworks/av/media/libcedarx/libcore/playback/player.c 文件。

```
diff --git a/media/libcedarx/libcore/playback/player.c b/media/libcedarx/libcore/playback/player.c
index 0e0baa0..304ae85 100644
--- a/media/libcedarx/libcore/playback/player.c
+++ b/media/libcedarx/libcore/playback/player.c
@@ -3415,7 +3415,7 @@ static int CallbackProcess(void* pSelf, int eMessageId, void* param)
        nCurTime  = p->pAvTimer->GetTime(p->pAvTimer);
        nTimeDiff = nVideoPts - nCurTime;

-       logv("notify video pts = %" PRId64 " ms, curTime = %" PRId64 " ms, diff = %"
+        logd("notify video pts = %" PRId64 " ms, curTime = %" PRId64 " ms, diff = %"
          PRId64 " ms", nVideoPts/1000, nCurTime/1000, nTimeDiff/1000);

        if (p->onResetNotSync)
@@ -3614,7 +3614,7 @@ static int CallbackProcess(void* pSelf, int eMessageId, void* param)

        }

-       logv("notify audio pts %" PRId64 " ms, curTime %" PRId64 " ms, diff %" PRId64
+        logd("notify audio pts %" PRId64 " ms, curTime %" PRId64 " ms, diff %" PRId64
          " ms, cacheTime %" PRId64 " ms", nAudioPts/1000, nCurTime/1000,
          nTimeDiff/1000, nCachedTimeInSoundDevice/1000);
```

分析打印，看看 Audio 和 Video 的 pts 是否正常。

2、查看 Audio/Video Decoder 是否正常

修改 android/frameworks/av/media/libcedarx/libcore/playback/audioDecComponent.c 文件以及 android/frameworks/av/media/libcedarx/libcore/playback/videoDecComponent.c 文件。

```
diff --git a/media/libcedarx/libcore/playback/audioDecComponent.c b/media/libcedarx/libcore/playback/
    audioDecComponent.c
index da72249..ab00299 100755
--- a/media/libcedarx/libcore/playback/audioDecComponent.c
+++ b/media/libcedarx/libcore/playback/audioDecComponent.c
@@ -1312,7 +1312,7 @@ static void doDecode(AwMessage *msg, void *arg)
                &p->pStreamInfoArr[p->nStreamSelected],
                pOutputBuf,
                &nPcmDataLen);
-   logv("DecodeAudioStream, ret = %d",ret);
+   logd("DecodeAudioStream, ret = %d",ret);
    if(ret == ERR_AUDIO_DEC_NONE)
    {
      if(p->pStreamInfoArr[p->nStreamSelected].nSampleRate != p->bsInfo.out_samplerate ||
diff --git a/media/libcedarx/libcore/playback/videoDecComponent.c b/media/libcedarx/libcore/playback/
    videoDecComponent.c
index 8b050ad..6bec2e4 100755
--- a/media/libcedarx/libcore/playback/videoDecComponent.c
+++ b/media/libcedarx/libcore/playback/videoDecComponent.c
@@ -820,7 +820,7 @@ static void doDecode(AwMessage *msg, void *arg)
                p->bConfigDropDelayFrames,
                nCurTime);

-   logv("DecodeVideoStream return = %d, p->bCrashFlag(%d)", ret, p->bCrashFlag);
+   logd("DecodeVideoStream return = %d, p->bCrashFlag(%d)", ret, p->bCrashFlag);

    if(ret == VDECODE_RESULT_NO_BITSTREAM)
    {
```

分析解码库的返回值意义：

- VDECODE_RESULT_FRAME_DECODED(1): 解码成功，输出了一帧图像；
- VDECODE_RESULT_CONTINUE(2): 码流被解码，但没有图像输出，需继续解码；
- VDECODE_RESULT_KEYFRAME_DECODED(3): 解码成功，输出了一帧关键帧图像；
- VDECODE_RESULT_NO_FRAME_BUFFER(4): 当前无法获取到图像 Buffer；
- VDECODE_RESULT_NO_BITSTREAM(5): 当前无法获取到码流数据；
- VDECODE_RESULT_RESOLUTION_CHANGE(6): 视频分辨率发生变化；
- VDECODE_RESULT_UNSUPPORTED(-1): 不能支持的格式或申请内存失败，无法继续解码；

3、查看 Demuxer 给解码库 submit Audio/Video 数据是否正常

android/frameworks/av/media/libcedarx/libcore/playback/player.c 文件。

```
diff --git a/media/libcedarx/libcore/playback/player.c b/media/libcedarx/libcore/playback/player.c
index 0e0baa0..7a2416c 100644
--- a/media/libcedarx/libcore/playback/player.c
+++ b/media/libcedarx/libcore/playback/player.c
@@ -1365,7 +1365,7 @@ int PlayerSubmitStreamData(Player*          pl,
```

```
  p = (PlayerContext*)pl;

-  logv("submit stream data, eMediaType = %d", eMediaType);
+  logd("submit stream data, eMediaType = %d", eMediaType);

  if(p->eStatus == PLAYER_STATUS_STOPPED)
```

根据这句打印，判断 Demuxer 后的音视频数据是否送给解码库，每笔数据的 PTS。其中 eMediaType 含义如下。

```
enum EMEDIATYPE
{
  MEDIA_TYPE_VIDEO = 0,
  MEDIA_TYPE_AUDIO,
  MEDIA_TYPE_SUBTITLE
};
```

经过以上三步分析，可以初步定位出播放异常问题属于 Render —> Decoder —> Demuxer （Audio/Video/Subtitle）哪个模块。

5.3 如何简单定位 MediaCodec 通路播放问题

MediaCodec 通路往下是通过 Codec2.0 的服务来对接实现编解码功能的，因此分析 MediaCodec 播放问题时，可以 Dump 出 Codec2.0 服务相关信息进行分析。全志的 Codec2.0 服务中，支持以设置系统属性的方式来进行动态调试，使用方式如下。

```
setprop persist.vendor.codec2.debug 调试模式
```

调试模式值的设置，目前支持以下几种：

```
enum CODEC_MODE {
 INPUT = 0, //编解码输入调试模式
 OUTPUT, //编解码输出调试模式
 PERFORMANCE, //编解码性能调试模式
 BUFFER_STATUS, //编解码buffer轮转状态调试模式
 DECODE_STATUS, //编解码结果调试模式
 CODEC_MODE_END,
};

enum DEBUG_MODE : std::int32_t {
 DEBUG_NONE, //0: 代表关闭所有调试
 DEBUG_DUMP_INPUT = 1 << INPUT, //0x01:保存编解码器输入数据
 DEBUG_DUMP_OUTPUT = 1 << OUTPUT, //0x02：保存编解码器输出数据
 DEBUG_DUMP_PERFORMANCE = 1 << PERFORMANCE, //0x04:打印解码器解码速度
 DEBUG_DUMP_BUFFER_STATUS = 1 << BUFFER_STATUS, //0x08:打印解码器buffer状态
 DEBUG_DUMP_DECODE_STATUS = 1 << DECODE_STATUS, //0x10:打印解码器解码输出返回结果
};
```

多种模式值或处理，代表同时打开多种调试模式，这样可以更全面的将调试信息输出后，提供给 FAE 进行下一步分析。如下例子为打开保存解码器输入数据和打印解码器速度的方式。

```
setprop persist.vendor.codec2.debug 0x05
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.4 如何保存解码前的码流和解码后的图片

修改小机端/vendor/etc/cedarc.conf 文件。

```
busybox vi /vendor/etc/cedarc.conf
```

vdecoder_save_bitstream 设置为 1，表示保存解码前的码流，文件路径为 vdecoder_save_bitstream_path。vdecoder_save_picture_en 设置为 1，表示保存解码后的 YUV 数据，文件路径为 vdecoder_save_picture_path。

```
# save bitstream in vdecoder.c
vdecoder_save_bitstream = 0 ##
vdecoder_save_bitstream_path = /data/camera/bs.dat

# save picture from start_num to start_num + total_num
vdecoder_save_picture_en = 0
vdecoder_save_picture_start_num = 0
vdecoder_save_picture_total_num = 10
vdecoder_save_picture_path = /data/camera/pic.da
```

解码出来的图片，根据格式使用 YUV 或 NV 工具查看。

可以同时将 cedarc.conf 中的 vdecoder_save_special_bitstream 打开，保存全志特殊格式码流，用于提供给 FAE 进行下一步分析。

```
vdecoder_save_special_bitstream = 0
vdecoder_save_special_bitstream_path = /data/camera/spec.awsp
```

注意：如果保存失败，请查看当前机子是否有"/data/camera/"目录，权限是否正确。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 为什么有些音视频不能播放

1、判断封装格式、流媒体协议是否支持。

2、判断音频/视频的 Codec 格式是否支持，有些音频/视频格式属于版权视频，需要查看支持列表看看是否支持。如果需要支持，请咨询 FAE。

3、判断视频 Codec 规格是否支持对应 size 格式播放，比如 Video 格式 H265 4K@30fps。

如果平台规格支持，又不能播放，可以尝试使用 5.2 章节的方法定位问题，反馈给 FAE。

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。